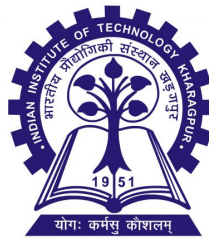


Dept. of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

VLSI Design Lab



Experiment-3A

Design and Implementation of N-bit Adders using Verilog

Chiradip Biswas 22EC37004

Arup Basak 22EC37021

1 Introduction

The objective of this experiment is to design and implement a fundamental 4-bit Carry Lookahead Adder (CLA) and utilize it as a building block to hierarchically construct larger adders, specifically 8-bit, 12-bit, and 16-bit architectures.

By employing a structural modeling approach, the larger adders ripple the carry signal between the smaller, high-speed 4-bit CLA blocks. The designs are simulated for functional verification and synthesized to analyze resource utilization (LUTs, IOBs) and timing constraints (Critical Path Delay) across varying bit-widths.

2 Verilog Implementation

2.1 4-Bit Carry Lookahead Adder (Base Module)

The 4-bit adder uses Carry Lookahead logic to calculate generate (g) and propagate (p) signals, computing all carry bits simultaneously to minimize delay.

```
1 module adder_4bit(  
2     input [3:0] a,  
3     input [3:0] b,  
4     input cin,  
5     output [3:0] sum,  
6     output cout  
7 );  
8  
9     wire [3:0] p;  
10    wire [3:0] g;  
11    wire [4:0] c;  
12  
13    assign p = a ^ b;  
14    assign g = a & b;  
15  
16    assign c[0] = cin;  
17    assign c[1] = g[0] | (p[0] & c[0]);  
18    assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0]);  
19    assign c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] &  
20    p[1] & p[0] & c[0]);  
21    assign c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] &  
22    p[2] & p[1] & g[0]) | (p[3] & p[2] & p[1] & p[0] & c[0]);  
23  
24    assign sum = p ^ c[3:0];  
25    assign cout = c[4];  
endmodule
```

Listing 1: 4-bit CLA Implementation

2.2 8-Bit, 12-Bit, and 16-Bit Structural Adders

Larger adders are built by instantiating the 4-bit CLA blocks and chaining the carry-out to the next block's carry-in.

```
1 module adder_8bit(  
2     input [7:0] a,  
3     input [7:0] b,  
4     input cin,  
5     output [7:0] sum,  
6     output cout  
7 );  
8  
9 wire c4;  
10  
11 adder_4bit lower_nibble (  
12     .a(a[3:0]),  
13     .b(b[3:0]),  
14     .cin(cin),  
15     .sum(sum[3:0]),  
16     .cout(c4)  
17 );  
18  
19 adder_4bit upper_nibble (  
20     .a(a[7:4]),  
21     .b(b[7:4]),  
22     .cin(c4),  
23     .sum(sum[7:4]),  
24     .cout(cout)  
25 );  
26 endmodule
```

Listing 2: 8-bit Adder

```
1 module adder_12bit(  
2     input [11:0] a,  
3     input [11:0] b,  
4     input cin,  
5     output [11:0] sum,  
6     output cout  
7 );  
8  
9 wire c1_2, c2_3;  
10  
11 adder_4bit lower_nibble (  
12     .a(a[3:0]), .b(b[3:0]), .cin(cin), .sum(sum[3:0]), .cout(c1_2)  
13 );  
14 adder_4bit mid_nibble (  
15     .a(a[7:4]), .b(b[7:4]), .cin(c1_2), .sum(sum[7:4]), .cout(c2_3)  
16 );  
17 adder_4bit upper_nibble (  
18     .a(a[11:8]), .b(b[11:8]), .cin(c2_3), .sum(sum[11:8]), .cout(cout)  
19 );
```

```

18     .a(a[11:8]), .b(b[11:8]), .cin(c2_3), .sum(sum[11:8]), .cout(
    cout)
19 );
20 endmodule

```

Listing 3: 12-bit Adder

```

1 module adder_16bit(
2     input [15:0] a,
3     input [15:0] b,
4     input cin,
5     output [15:0] sum,
6     output cout
7 );
8
9 wire c4;
10
11 adder_8bit lower_nibble (
12     .a(a[7:0]), .b(b[7:0]), .cin(cin), .sum(sum[7:0]), .cout(c4)
13 );
14
15 adder_4bit upper_nibble (
16     .a(a[15:8]), .b(b[15:8]), .cin(c4), .sum(sum[15:8]), .cout(cout
17 )
18 );
19 endmodule

```

Listing 4: 16-bit Adder

2.3 Testbench Implementations

To verify the functionality of the adders, stimulus was applied using automated testbenches. The `$random` system task was utilized to generate a variety of input combinations across the full valid range for each bit-width to ensure robust verification.

```

1 module adder_tb;
2     reg [3:0] a,b;
3     wire [3:0] sum;
4     wire c;
5
6     adder_4bit uut(a,b,0,sum,c);
7
8     initial begin
9         #5 a=7; b=1;
10        repeat(10) begin
11            #5 a={$random}%16; b={$random}%16;
12        end
13        #50 $finish;
14    end
15 endmodule

```

Listing 5: 4-bit Adder Testbench

```

1 module adder_8bit_tb;
2     reg [7:0] a,b;
3     wire [7:0] sum;
4     wire c_out;
5
6     adder_8bit uut(a,b,0,sum,c_out);
7
8     initial begin
9         #5 a=15; b=1;
10        repeat(10) begin
11            #5 a={$random}%256; b={$random}%256;
12        end
13        #50 $finish;
14    end
15 endmodule

```

Listing 6: 8-bit Adder Testbench

```

1 module adder_12bit_tb;
2     reg [11:0] a,b;
3     wire [11:0] sum;
4     wire c_out;
5
6     adder_12bit uut(a,b,0,sum,c_out);
7
8     initial begin
9         #5 a=15; b=1;
10        repeat(15) begin
11            #5 a={$random}%4096; b={$random}%4096;
12        end
13        #50 $finish;
14    end
15 endmodule

```

Listing 7: 12-bit Adder Testbench

```

1 module adder_16bit_tb;
2     reg [15:0] a,b;
3     wire [15:0] sum;
4     wire c_out;
5
6     adder_16bit uut(a,b,0,sum,c_out);
7
8     initial begin
9         #5 a=15; b=1;
10        repeat(20) begin
11            #5 a={$random}%65536; b={$random}%65536;
12        end
13        #50 $finish;
14    end

```

Listing 8: 16-bit Adder Testbench

3 Simulation Results

Post-implementation timing simulations were performed using testbenches that applied random inputs to the adders. The resulting waveforms demonstrate correct arithmetic operation and carry propagation across all stages.

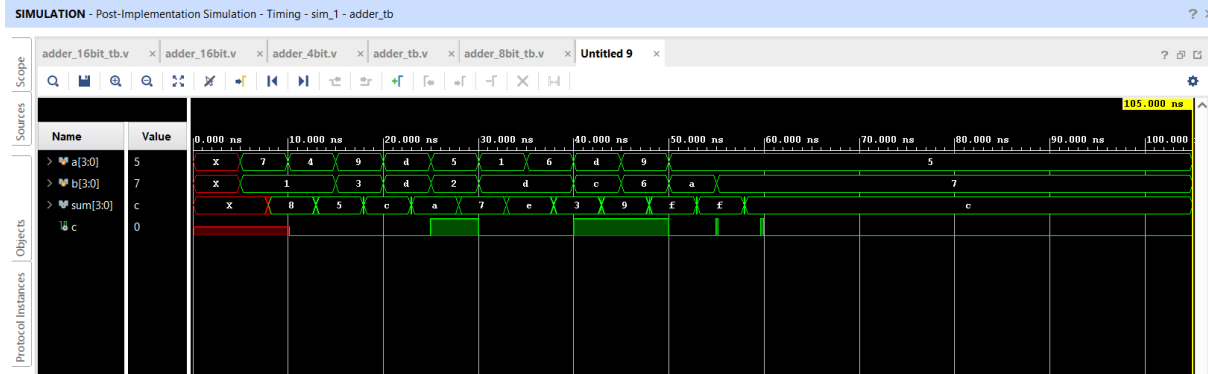


Figure 1: Post-Implementation Simulation Waveform for 4-bit Adder

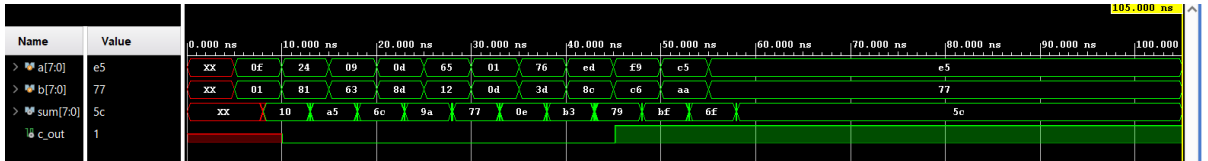


Figure 2: Post-Implementation Simulation Waveform for 8-bit Adder

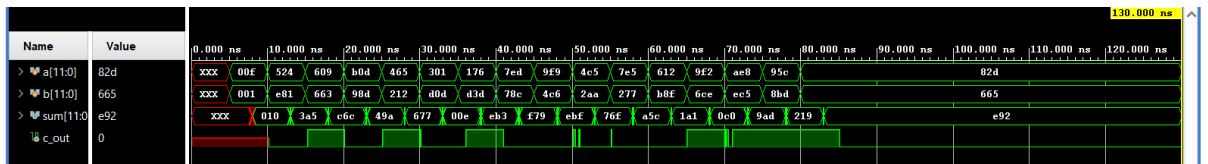


Figure 3: Post-Implementation Simulation Waveform for 12-bit Adder

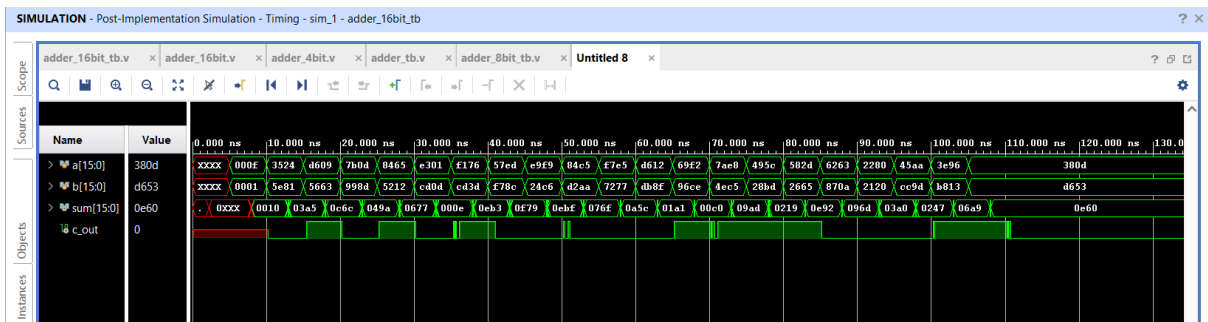


Figure 4: Post-Implementation Simulation Waveform for 16-bit Adder

4 Synthesis Results

The designs were synthesized using Xilinx Vivado targeting the xc7k70tfbg484-3 device.

4.1 Resource Utilization and Timing Data

Table 1 presents the comparative results extracted from the synthesis reports, showing resource consumption and the critical path delay as the adder width increases.

Table 1: Comparison of Resource Utilization and Timing across N-bit Adders

Design Module	Slice LUTs	Slice Count	Bonded IOB	Critical Path Delay (ns)
adder_4bit	4	-	14	4.159
adder_8bit	8	3	26	6.943
adder_12bit	20	6	38	7.739
adder_16bit	20	6	42	7.742

5 Discussions

Arup Basak 22EC37021

The structural implementation of varying bit-width adders using a base 4-bit Carry Lookahead Adder (CLA) provides clear insights into the trade-offs between hierarchical design, resource utilization, and combinational delay.

- **Timing and Critical Path Analysis:** As observed in the synthesis reports, the critical path delay increases with the bit-width of the adder. The 4-bit adder achieves a fast propagation delay of **4.159 ns** because its carry bits are computed simultaneously using CLA logic. However, when cascading these blocks to form the 8-bit (**6.943 ns**), 12-bit (**7.739 ns**), and 16-bit (**7.742 ns**) adders, the carry must "ripple" from the lower block to the upper blocks. This block-ripple architecture creates a longer critical path, confirming that while intra-block computation is fast, inter-block carry propagation becomes the timing bottleneck for wider adders. Interestingly, the delay difference between the 12-bit and 16-bit adder is negligible (0.003 ns), suggesting the synthesis tool optimized the routing paths heavily.
- **Resource Utilization Scaling:** The hardware resource consumption scales largely as expected with the input width. The 4-bit adder utilizes 4 Slice LUTs, the 8-bit uses 8 LUTs, and the 12-bit and 16-bit both synthesized to 20 LUTs. The Bonded IOB count increases precisely to accommodate the expanding input and output ports, capping at 42 IOBs for the 16-bit design.
- **Structural Reusability:** Building the 8-bit, 12-bit, and 16-bit adders purely by instantiating the 4-bit module demonstrates excellent code reusability. By using smaller, verified structural blocks, complex hierarchical designs can be assembled efficiently while maintaining readability.

Chiradip Biswas 22EC37004

- In the experiment, we made a 8bit adder using 2 4bit carry look ahead adders connected in ripple carry adder configuration. Then made a 12bit adder by cascading the 4bit adders in ripple carry manner.

5.1 Carry Lookahead Adder (CLA)

A Carry Look-ahead Adder improves speed by calculating the carry bits in advance based on the input signals, mitigating the delay caused by the traditional ripple effect. For each bit position i , two intermediate signals are defined: Generate (g_i) and Propagate (p_i).

- **Generate (g_i):** A carry is generated if both inputs a_i and b_i are 1.

$$g_i = a_i \cdot b_i \quad (1)$$

- **Propagate (p_i):** A carry is propagated to the next bit if at least one of the inputs is 1. In modern CLAs, this is typically implemented using the XOR operation:

$$p_i = a_i \oplus b_i \quad (2)$$

The sum (s_i) and the next carry out (c_{i+1}) are defined as:

$$s_i = p_i \oplus c_i \quad (3)$$

$$c_{i+1} = g_i + (p_i \cdot c_i) \quad (4)$$

By recursively substituting the carry equations, we can calculate any carry bit directly from the initial carry-in (c_0) without waiting for previous bit additions to complete. For a 4-bit CLA block, the expanded carry equations are:

$$c_1 = g_0 + (p_0 \cdot c_0) \quad (5)$$

$$c_2 = g_1 + (p_1 \cdot g_0) + (p_1 \cdot p_0 \cdot c_0) \quad (6)$$

$$c_3 = g_2 + (p_2 \cdot g_1) + (p_2 \cdot p_1 \cdot g_0) + (p_2 \cdot p_1 \cdot p_0 \cdot c_0) \quad (7)$$

$$c_4 = g_3 + (p_3 \cdot g_2) + (p_3 \cdot p_2 \cdot g_1) + (p_3 \cdot p_2 \cdot p_1 \cdot g_0) + (p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0) \quad (8)$$

The final carry out (c_{out}) of the 4-bit block is c_4 .

5.2 Ripple Carry Architecture (8-bit Adder)

The 8-bit adder module connects two of these blocks using a Ripple Carry approach.

- **Lower Nibble:** Calculates the sum for bits 0–3 and generates an intermediate carry out, denoted as c_4 .
- **Upper Nibble:** Calculates the sum for bits 4–7. This block is dependent on the lower nibble and must wait for its computation to finish, as c_4 is passed in as the initial carry-in (c_{in}) for this stage.
- **Hierarchical Code Reusability:** Constructing the 8-bit, 12-bit, and 16-bit architectures through the direct instantiation of the foundational 4-bit module highlights the benefits of modular design.
- **Hardware Resource Scaling:** As anticipated, logic consumption grows in tandem with operand width. The synthesis mapped the 4-bit and 8-bit designs to 4 and 8 Slice LUTs, respectively, while the 12-bit and 16-bit configurations both required 20 LUTs. Additionally, the allocation of Bonded IOBs correctly expanded to handle the wider data buses, reaching a maximum of 42 IOBs for the 16-bit implementation.
- **Critical Path and Propagation Delay:** Synthesis reports confirm that wider adders suffer from increased critical path delays. The standalone 4-bit module evaluates rapidly in 4.159 ns

due to its internal Carry Lookahead (CLA) logic. Conversely, chaining these blocks to construct 8-bit (6.943 ns), 12-bit (7.739 ns), and 16-bit (7.742 ns) versions introduces a block-level ripple effect. This demonstrates that inter-block carry propagation forms the primary timing bottleneck, overshadowing the fast intra-block CLA computation. Notably, the minuscule delay variance (0.003 ns) between the 12-bit and 16-bit adders points to routing optimizations performed by the synthesis engine.